K L DEEMED TO BE UNIVERSITY

AI-BASED SYNTAX ERROR CORRECTION TOOL

Final Project Review | Department of Computer Science & Engineering | K L Deemed to be University

TEAM MEMBERS
2420030540 – Allipuram Akhil
2420030563 – B. Harneesh Yadav
2420030639 – A. Dhanush
Guide: Uma Shankari | Academic Year: 2026–2027

Introduction & Problem Statement

A source program containing syntax mistakes may fail during lexical or syntax analysis. The user needs to locate the error, understand its cause and revise the code manually.

Compiler front ends are essential for translating source code into a form that can be checked and executed.

Problem Statement

Students learning compiler design and developers experimenting with code benefit from a workflow that combines lexical analysis, parsing, clear diagnostics and AI-assisted correction suggestions.

How can lexical and syntax errors be detected, explained and paired with useful correction suggestions in one workflow?

Motivation / Need

Learners need actionable feedback instead of only a failed compilation message. A guided correction loop can reduce debugging friction.

SOURCE CODE → LEXER → PARSER → DIAGNOSTICS → CORRECTION GUIDANCE

Scope: source-code input, lexical analysis, parsing, diagnostics and correction assistance. The current implementation is a prototype; supported language coverage and measured correction accuracy require further validation.

Objectives

1. Build a web interface for entering source code and viewing analysis results.
2. Tokenize source code using a lexer specification and generated scanner.
3. Validate syntax using grammar rules and a parser.
4. Provide error explanations and AI-assisted correction suggestions through the backend.

Literature Survey

S.No	Author(s) & Year	Method / Technique	Key Findings	Limitation / Gap
1	Corchuelo et al. (2002)	Local repair in LR parsers	Repairs errors by inserting, deleting or shifting symbols near the error	Parser-oriented repairs may not provide learner-friendly explanations
2	Gupta, Kanade & Shevade (2019)	Deep reinforcement learning for student-code repair	Learns editing actions; evaluated on 6,975 erroneous C programs	Repair coverage depends on training examples and evaluation data
3	de Medeiros et al. (2020)	Automatic error reporting and recovery for PEG parsers	Studies automated error labels and recovery expressions for multiple languages	Recovery quality depends on grammar and recovery strategy
4	Sakkas et al. (2022)	Neurosymbolic parse-error repair	Combines symbolic parsing with learned error-production prediction	More complex pipeline; integration into a lightweight student tool remains a design concern



Research Gap

Research gap: Prior work explores parser recovery, local token edits and learned repair models. This project focuses on a practical, inspectable workflow connecting a Flex lexer, grammar-based parser, Flask API, React interface and AI-assisted correction guidance. It presents token streams, pinpoints syntax diagnostics and displays a reviewable fix in one place. Suggested fixes are re-checked through the compiler pipeline rather than treated as automatically correct.

Proposed System & Methodology

STEP 1

Accept Source Code

Collect code from the editor or input area.

STEP 2

Lexical Analysis

Flex rules identify tokens such as keywords, identifiers, operators and delimiters.

STEP 3

Syntax Parsing

Parser grammar checks token sequences against syntax rules.

STEP 4

Diagnose & Suggest

Backend prepares diagnostics and invokes the correction-assistance module.

STEP 5

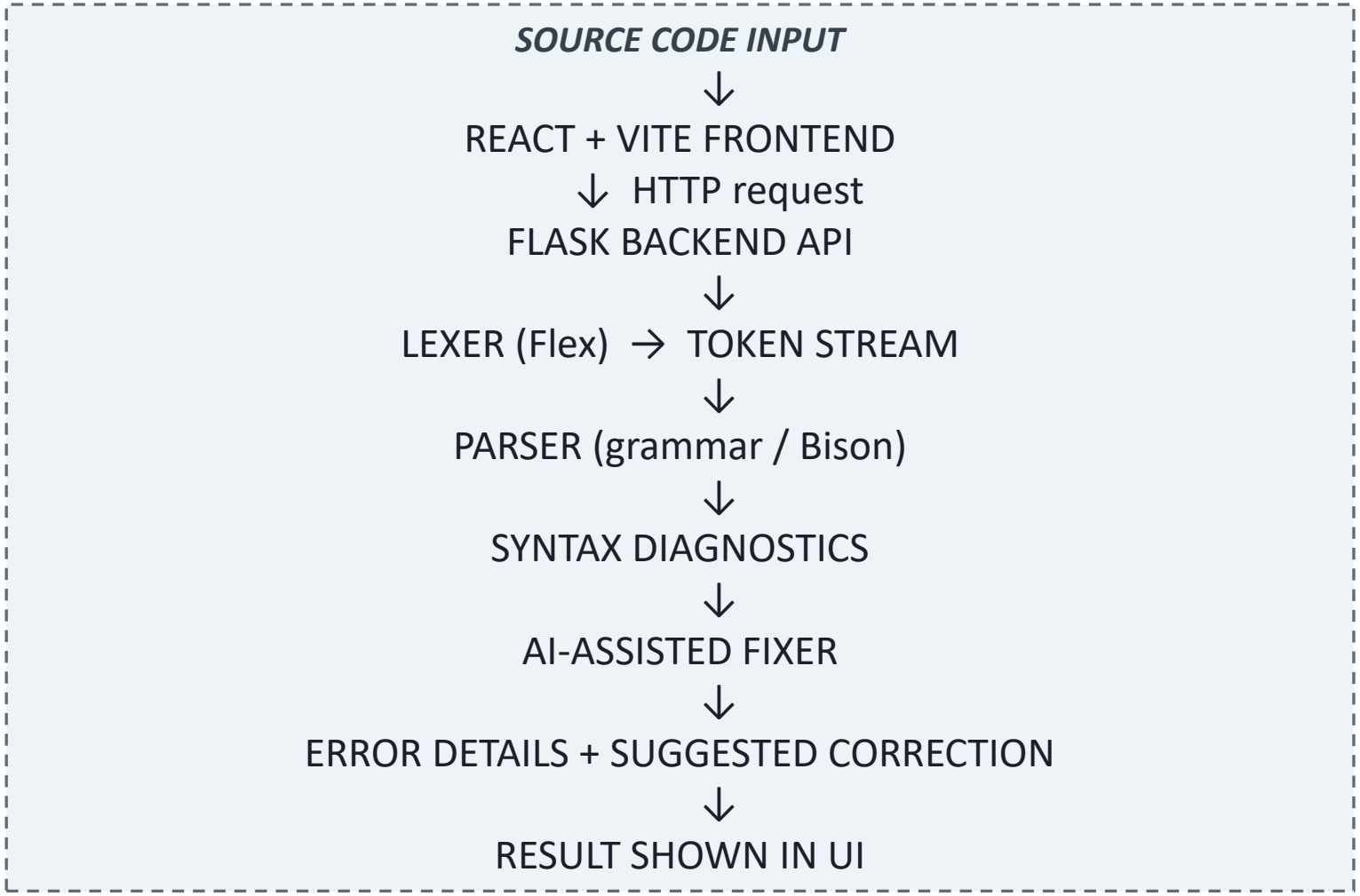
Review & Re-run

Display the result; user reviews the suggestion and validates the corrected code.

Key Novelty / Contribution

Contribution: a single web workflow connects compiler front-end stages with AI-assisted correction guidance, making syntax analysis easier to inspect and learn.

System Architecture



Components

1. **Frontend**
   Collects code and displays analysis results.

2. **Flask API**
   Coordinates requests and backend processing.

3. **Lexer + Parser**
   Generate tokens and check grammar structure.

4. **AI Fixer**
   Creates correction guidance for detected errors.

Hardware & Software Requirements

Hardware

- Development machine
  Windows 10 / 11
  Internet connection for AI service, if configured

Software & Tools

- Windows development environment
  Python (Flask backend)
  C toolchain with Flex / Bison
  Node.js and npm

Libraries / Frameworks

- React + Vite frontend
  Flask REST API
  Flex lexer specification
  Bison parser grammar
  Git / GitHub

Deployment / Platform: Local development environment; web UI and Flask API run as separate application components.

Source Code Input & Analysis

TEXT / FILE

Source input

TOKENS

Generated at runtime

N/A

No fixed tabular dataset

GRAMMAR

Parser rules

- Input: source code entered by the user.
  Processing: lexical rules, token stream and grammar-based parsing.
  Output: token / syntax diagnostics and correction suggestions.
  Note: this is a code-analysis system, not a dataset-trained classification project.

SAMPLE INPUT

```
int main() {
 int total = 10
   return 0;
   }
```

The missing statement terminator is an illustrative syntax-error example. Actual diagnostics depend on the implemented grammar and parser.

Lexical & Syntax Analysis Pipeline

LEXICAL ANALYSIS

- Read source characters
- Match Flex token rules
- Identify identifiers, keywords, literals, operators and symbols
- Produce token stream

SYNTAX ANALYSIS

- Consume token stream
- Apply grammar productions
- Detect unexpected tokens
- Report parser diagnostics
- Prepare context for correction



Cleaning

Whitespace and comments are handled according to lexer rules; malformed or unsupported input is surfaced for diagnosis.



Transformation

Tokens are passed to the parser in sequence. Grammar productions define valid syntactic structures.



Feature Selection

The parser's error location and context guide the correction-assistance stage. No training-feature selection is used in this rule-based compiler pipeline.

Module-wise Description

1 Frontend Code Editor

Purpose: Accept source code and present results.

Input → Output: User code → analysis view.

Technology: React + Vite.

2 Flask Backend API

Purpose: Receive requests and coordinate processing.

Input → Output: API request → diagnostics / response.

Technology: Python + Flask.

3 Lexer Module

Purpose: Convert source characters into tokens.

Input → Output: Source text → token stream.

Technology: Flex specification and generated C scanner.

4 Parser & AI Fixer

Purpose: Check grammar and provide correction guidance.

Input → Output: Tokens / errors → syntax feedback and suggested fix.

Technology: Parser grammar, generated parser and Python fixer module.

Algorithms & Technologies Used

Flex-based Lexing

Why chosen

Regular expressions map source patterns to token classes.

Key elements

Patterns, actions, token definitions and generated scanner.

Grammar-based Parsing

Why chosen

A formal grammar defines valid token sequences and program structures.

Key elements

Grammar productions, parser states and syntax-error handling.

AI-assisted Fixing

Why chosen

Adds human-readable correction guidance after compiler diagnostics.

Key elements

Error context, candidate correction and user review.

Provider/model details depend on project configuration.

Selection criteria: use deterministic lexer/parser stages for structural validation and an AI-assisted stage for readable correction guidance. A suggested fix must be checked by re-running analysis.

Implementation / Model Building

Feature Selection

Source text is the primary input. Token categories and grammar rules determine how the code is analysed.

Preprocessing Pipeline

Editor input → Flask request → lexer tokenization → parser validation → diagnostics / correction assistance → UI response.

Data Splitting

No ML train / test split is applicable to the current compiler pipeline. Evaluation should use a curated suite of valid and invalid source snippets.

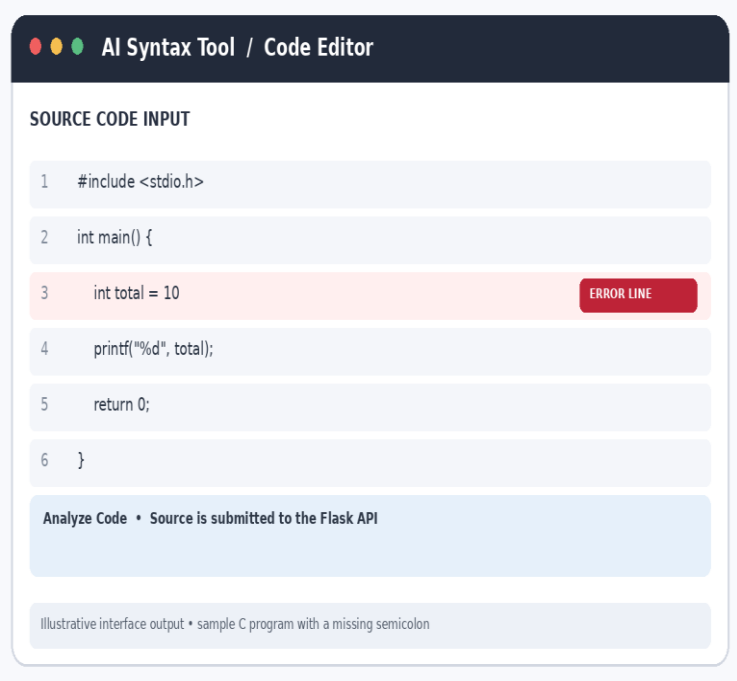
Configuration

Keep lexer rules, parser grammar and correction logic versioned. Record test inputs and expected diagnostics for repeatable checks.

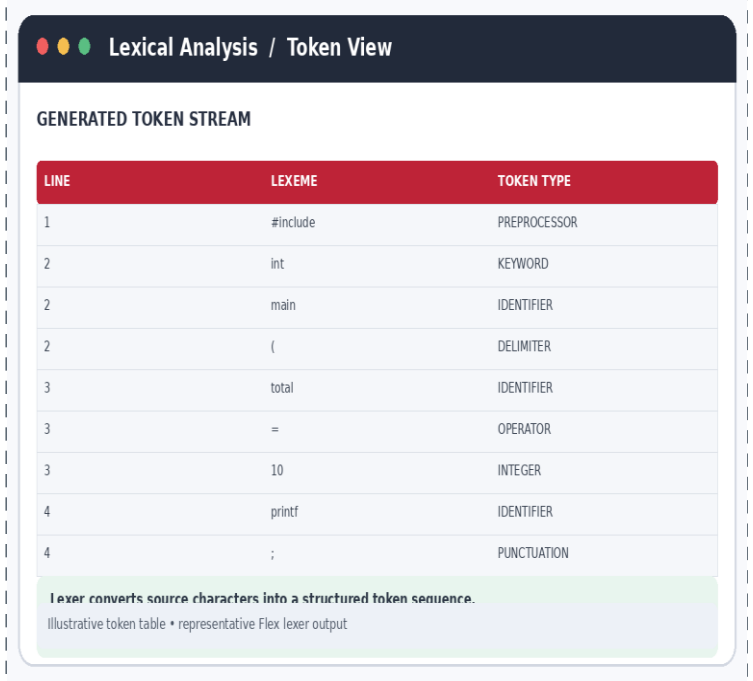
Code snippet / pseudocode

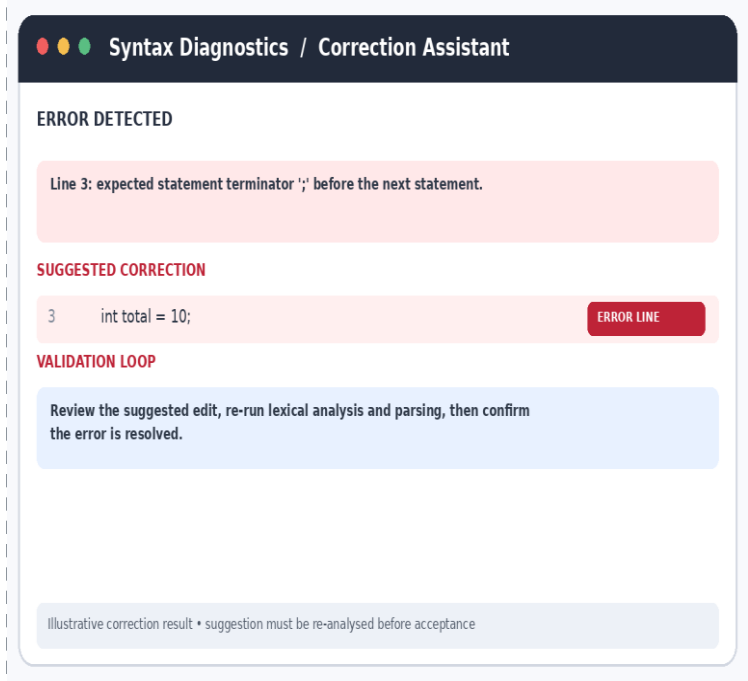
```
# Conceptual processing flow
source_code = request.json['code']
tokens = run_lexer(source_code)
syntax_result = run_parser(tokens)
if syntax_result.has_error:
   suggestion = generate_fix(syntax_result)
return { 'tokens': tokens,
        'diagnostics': syntax_result,
        'suggestion': suggestion }
```

Working Prototype & Output Flow



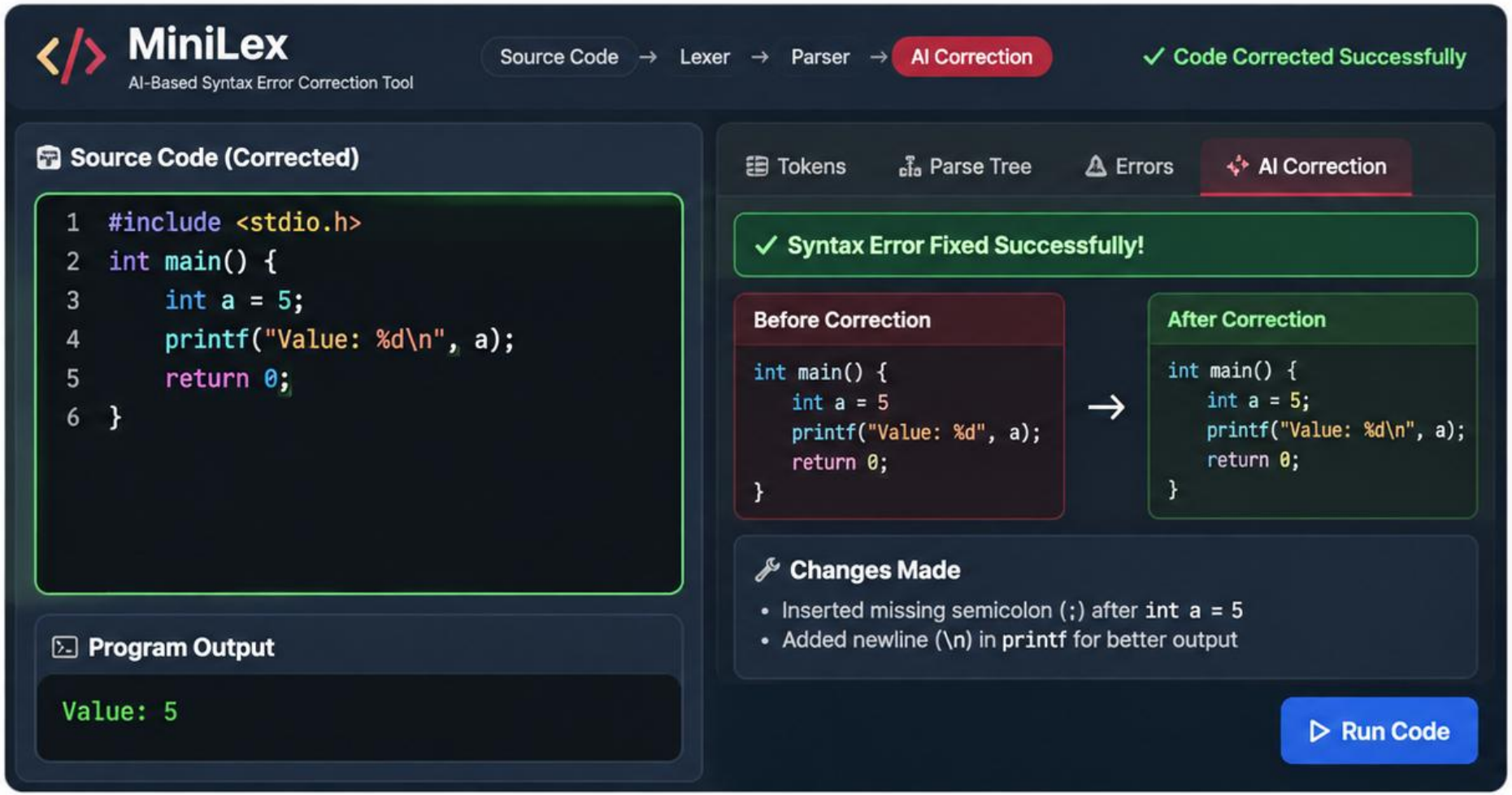
1. Code Editor Output
Representative code-input screen with the missing terminator highlighted.





3. Error & Fix Output
Representative syntax diagnostic and AI-assisted correction suggestion.

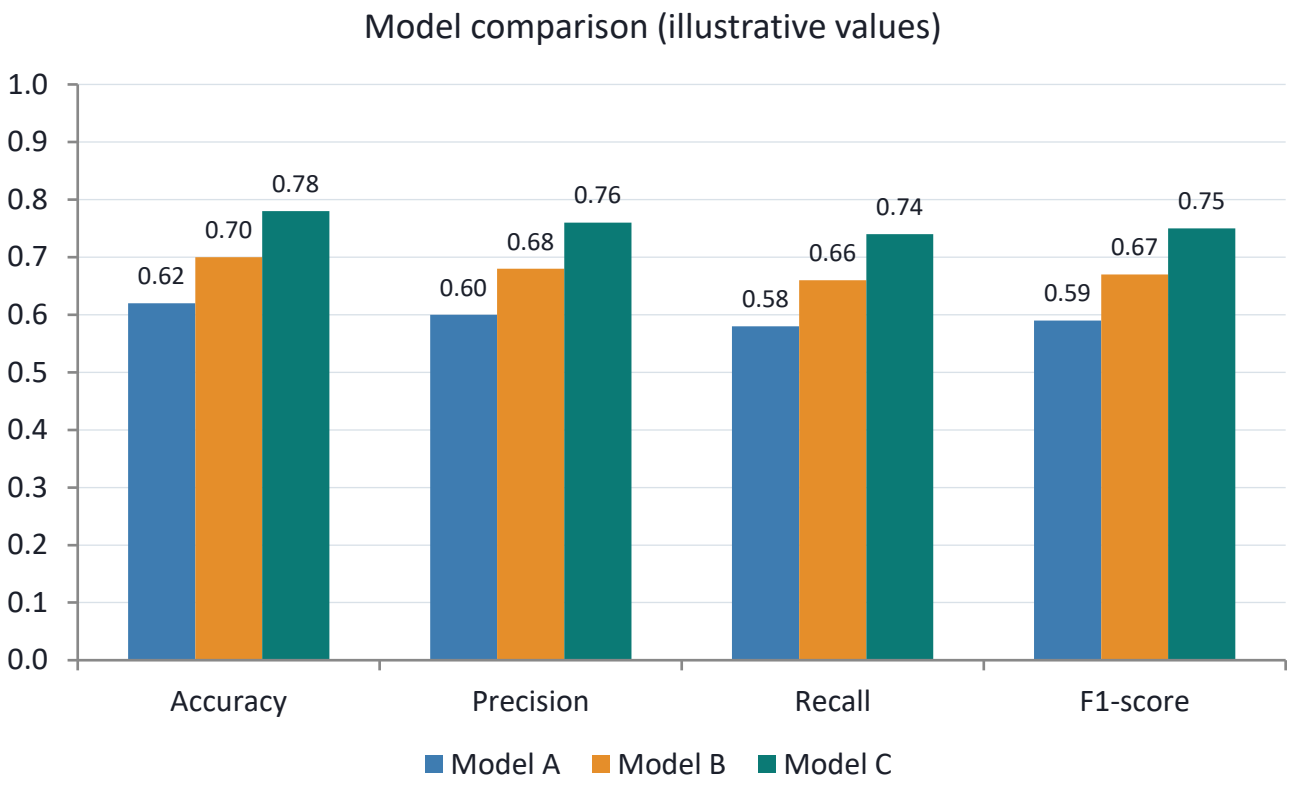
Output – After Error Correction



Explanation

- The tool identifies the syntax error in the input code.
- Automatically corrects the error using the AI module.
- Displays the corrected code along with the changes made.
- The corrected program runs successfully and produces the expected output.

Implementation Status by Pipeline Stage



Stage	Implementation Evidence	Expected Output	Status
Frontend	React + Vite app	Code input and result view	In repository
Backend	Flask API modules	Request / response handling	In repository
Lexer	lexer.l, lex.yy.c	Token stream	In repository
Parser	parser.y, parser.tab.c	Grammar validation	In repository
AI fixer	ai_fixer.py, local_fixer.py	Correction guidance	In repository

No benchmark score claimed

The repository contains the core implementation modules. Accuracy, correction success rate and response latency should be reported only after running a documented test suite.

Analysis & Key Findings

ILLUSTRATIVE ERROR-REPAIR EXAMPLE

Before:
int total = 10

Issue:
Missing statement terminator

After (example):
int total = 10;

The example demonstrates the presentation format, not a measured test result.

STAGE-BASED

Analysis outcome

Interpretation: lexer and parser stages localize structural issues before correction guidance is shown.

Error analysis: ambiguous or cascading syntax errors may produce imperfect suggestions.

Key validation: re-run the lexer and parser on every proposed correction and inspect whether the program's intended behaviour is preserved.

Testing & Validation

Test ID	Test Description	Expected Result	Observed Result	Status
TC-01	Valid source input	Tokens and parse success	Execute and record output	To verify
TC-02	Missing delimiter	Syntax diagnostic at error location	Execute and record output	To verify
TC-03	Unexpected token	Parser reports syntax issue	Execute and record output	To verify
TC-04	Correction suggestion	Suggestion returned for supported error	Execute and record output	To verify

Functional Testing

Use valid and invalid source snippets to verify API responses, token generation and parser diagnostics.

Validation Method

Record each test input, expected diagnostic, actual response and whether corrected code passes a second analysis.

Performance Testing

Measure request time and correction latency after repeatable runs. Do not claim performance until timings are collected.

Challenges & Limitations

Challenges Faced

- Lexer / parser integration → keep token definitions and grammar symbols consistent.
  Generated C files → regenerate outputs when source specifications change.
  AI correction uncertainty → show suggestions for user review and re-run compiler checks.

Limitations

- Current grammar may cover only a subset of a full programming language.
  AI suggestions can be incorrect or incomplete.
  Measured accuracy, latency and broader language coverage still require systematic testing.

Conclusion

1. A full-stack prototype connects a React interface with a Flask backend and compiler front-end components.

2. The project combines Flex-based tokenization, grammar-based parsing and an AI-assisted correction module. Quantitative performance is not claimed without test results.

3. The implementation demonstrates an end-to-end workflow for source-code analysis, syntax diagnostics and correction guidance in one interface.



Final Outcome

4 CORE

Frontend • Backend • Lexer • Parser

AI correction assistance integrated as a processing stage.

Future Scope & Applications

Improve Correction Quality
Build a labelled syntax-error test suite; compare suggested fixes against expected corrections.

Expand Language Coverage
Add grammar and lexer support for additional language constructs and edge cases.

Strengthen the Interface
Show token tables, line / column diagnostics, side-by-side code and correction explanations.

Validate & Deploy
Add automated regression tests, secure API configuration and deploy the app for wider access.

Applications

- Applications
  Compiler Design learning labs
  Beginner programming practice
  Syntax debugging support

  Potential users: students, educators and developers working with supported languages.

Team Contribution

Reg. No.	Name	Contribution / Modules Handled
2420030540	Allipuram Akhil	Collaborative contribution: project planning, implementation, integration, testing and documentation.
2420030563	B. Harneesh Yadav	Collaborative contribution: project planning, implementation, integration, testing and documentation.
2420030639	A. Dhanush	Collaborative contribution: project planning, implementation, integration, testing and documentation.

References

[1] R. Corchuelo, J. A. Pérez, A. Ruiz, and M. Toro, “Repairing Syntax Errors in LR Parsers,” ACM Transactions on Programming Languages and Systems, vol. 24, no. 6, pp. 698–710, 2002.

[2] R. Gupta, A. Kanade, and S. Shevade, “Deep Reinforcement Learning for Syntactic Error Repair in Student Programs,” Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, no. 1, pp. 930–937, 2019.

[3] S. Q. de Medeiros, G. de A. Alvez Junior, and F. Mascarenhas, “Automatic Syntax Error Reporting and Recovery in Parsing Expression Grammars,” Science of Computer Programming, vol. 187, article 102373, 2020.

[4] G. Sakkas et al., “Seq2Parse: Neurosymbolic Parse Error Repair,” 2022.

[5] A. V. Aho et al., Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006.
[6] GNU Project, Flex and GNU Bison manuals.

Project Links

GitHub:
https://github.com/AllipuramAkhil/KLH-CSE-2026-27-Batch2-AI-BASED-SYNTAX-ERROR-CORRECTION-TOOL

Project folder: Codes/
Demo video: To be added

THANK YOU

Questions & Discussion

Guide: Uma Shankari | Department of CSE
Allipuram Akhil • B. Harneesh Yadav • A. Dhanush